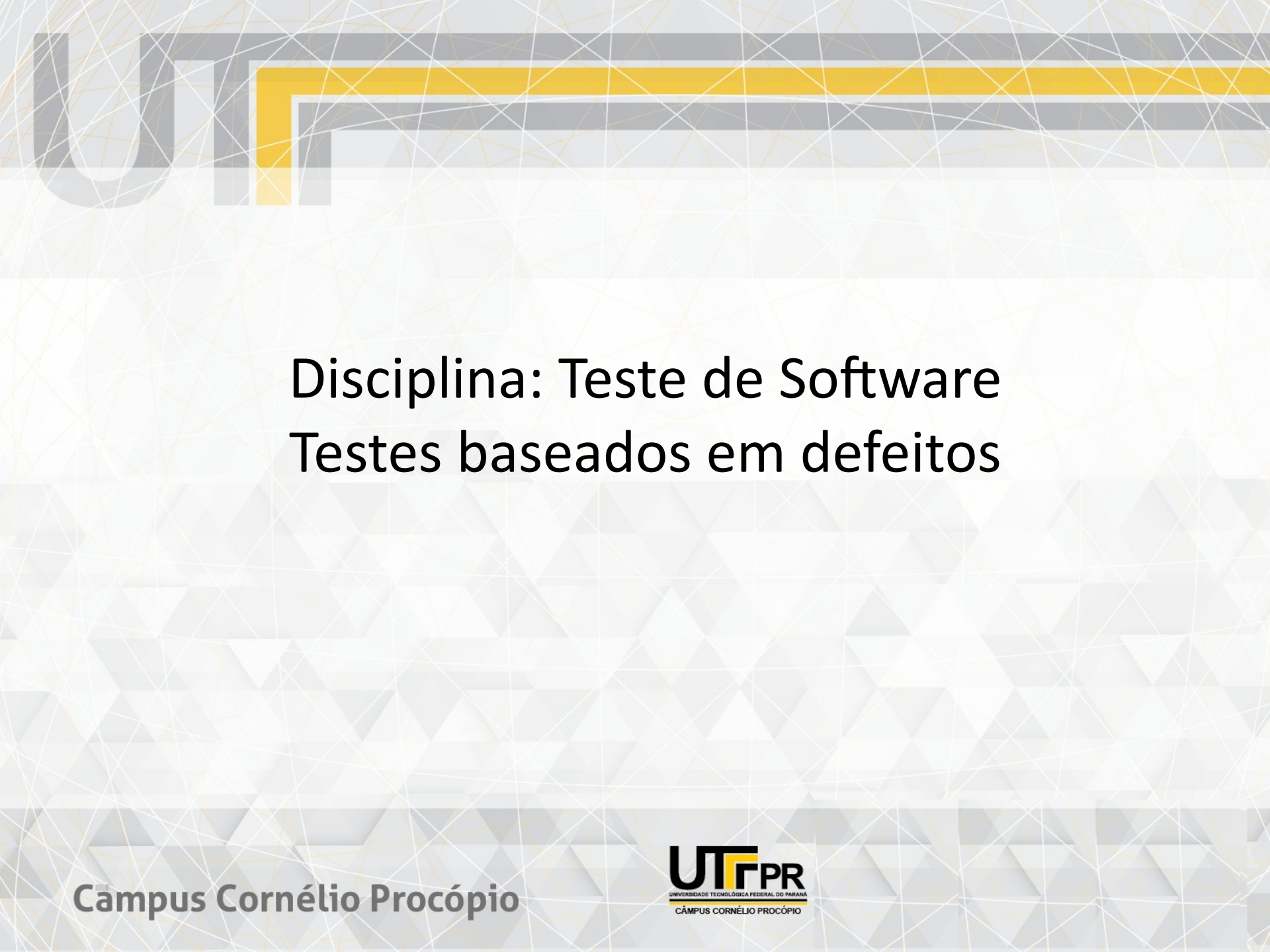




Disciplina: Teste de Software

Testes baseados em defeitos

Teste baseado em defeitos

- Utilização de informação sobre defeitos comuns
- Teste de mutação
- Semeadura de defeitos

Teste baseado em defeitos

- Análise de mutação
 - Hipótese do programador competente
 - Programadores experientes escrevem programas corretos ou muito próximos do correto
- Efeito do acoplamento
 - Casos de teste capazes de revelar falhas causadas por defeitos simples são tão sensíveis que, implicitamente, também são capazes de revelar falhas causadas por defeitos mais complexos

Teste baseado em defeitos

- Geração de mutantes
 - Mutantes são versões modificadas de um software, possivelmente com defeitos
- Verificar se uma abordagem, ferramenta ou técnica de teste de software, **especialmente** o conjunto de casos de teste, são efetivos
- Se o caso de teste ou abordagem, ferramenta e técnica para teste de software revela falha em um mutante com defeito, é dito que o mutante foi morto

Teste baseado em defeitos

- Geração de mutantes
 - Um número grande de mutantes pode ser gerado
 - Mutantes podem sobreviver
 - Mutantes que se comportam como o software original (mesmo resultado) são denominados mutantes equivalentes

Teste baseado em defeitos

- Mutantes equivalentes: quando a modificação realizada no software original para criar um mutante não altera a funcionalidade implementada, isto é, o software mutante funciona da mesma forma que o software original

Comando original	Mutante equivalente
<code>numero = 0;</code>	<code>numero *= 0;</code>
<code>for (i = 0; i < n; i++)</code>	<code>for (i = 0; i != n; i++)</code>
<code>if ((a > b) (c < d))</code>	<code>if ((a > b) + (c < d))</code>

Teste baseado em defeitos

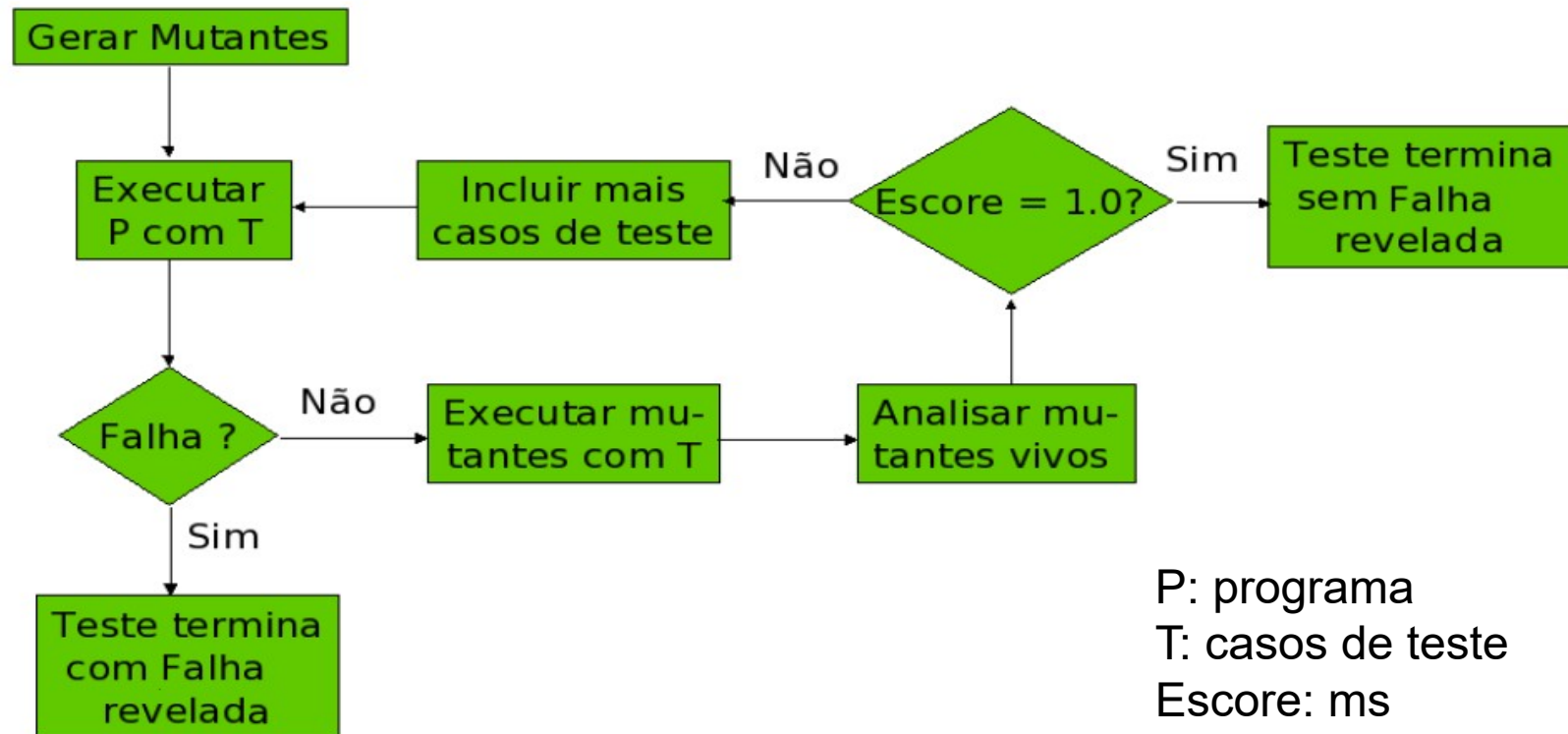
- Passo a passo para o teste de mutação
 - Geração dos mutantes a partir dos operadores de mutação selecionados e do software original
 - Execução de um conjunto de teste contra o software original. Caso as saídas geradas estejam de acordo com as saídas esperadas, esse conjunto de teste é executado contra os mutantes

Teste baseado em defeitos

- Passo a passo para o teste de mutação (continuação)
 - Identificação dos mutantes mortos e sobreviventes
 - Análise dos mutantes sobreviventes, incluindo a verificação dos mutantes equivalentes
 - Aplicação do escore de mutação

Teste baseado em defeitos

- Passo a passo para o teste de mutação (Delamaro)



Teste baseado em defeitos

- Ferramentas para testes de mutação
 - Mothra: software escrito em Fortran
 - Proteum: software escrito em C
 - Jumble: software escrito em Java
 - JesTer: software escrito em Java
 - muJava: software escrito em Java
 - MuClipse: desenvolvida a partir da muJava para o ambiente de desenvolvimento Eclipse

Teste baseado em defeitos

- muJava
 - Sistemas escritos usando a linguagem de programação Java
 - Página: <https://www.albany.edu/faculty/offutt/mujava/>
 - Visão geral
 - Download
 - Configuração
 - Uso da ferramenta
 - Publicações

Teste baseado em defeitos

- muJava
- Download
 - Windows
 - Linux
 - Mac

Teste baseado em defeitos

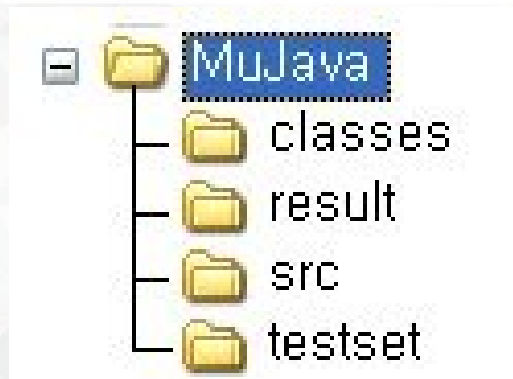
- muJava
- Configuração
 - Variáveis de ambiente (considerando o diretório C:\mujava)
 - Comando no Prompt de Comando
 - set CLASSPATH=%CLASSPATH%;C:\mujava\mujava.jar;C:\mujava\openjava.jar;C:\Program Files\Java\jdk1.7.0\lib\tools.jar;C:\mujava\classes
 - Depende do local de instalação do JDK

Teste baseado em defeitos

- muJava
 - Download
 - Windows
 - Linux
 - Mac
 - Configuração
 - Estrutura de diretórios (src, result, classes, testset)
 - Comando no Prompt de Comando
 - *java mujava.makeMuJavaStructure*

Teste baseado em defeitos

- muJava
- Configuração
 - Estrutura de diretórios (src, result, classes, testset)



Teste baseado em defeitos

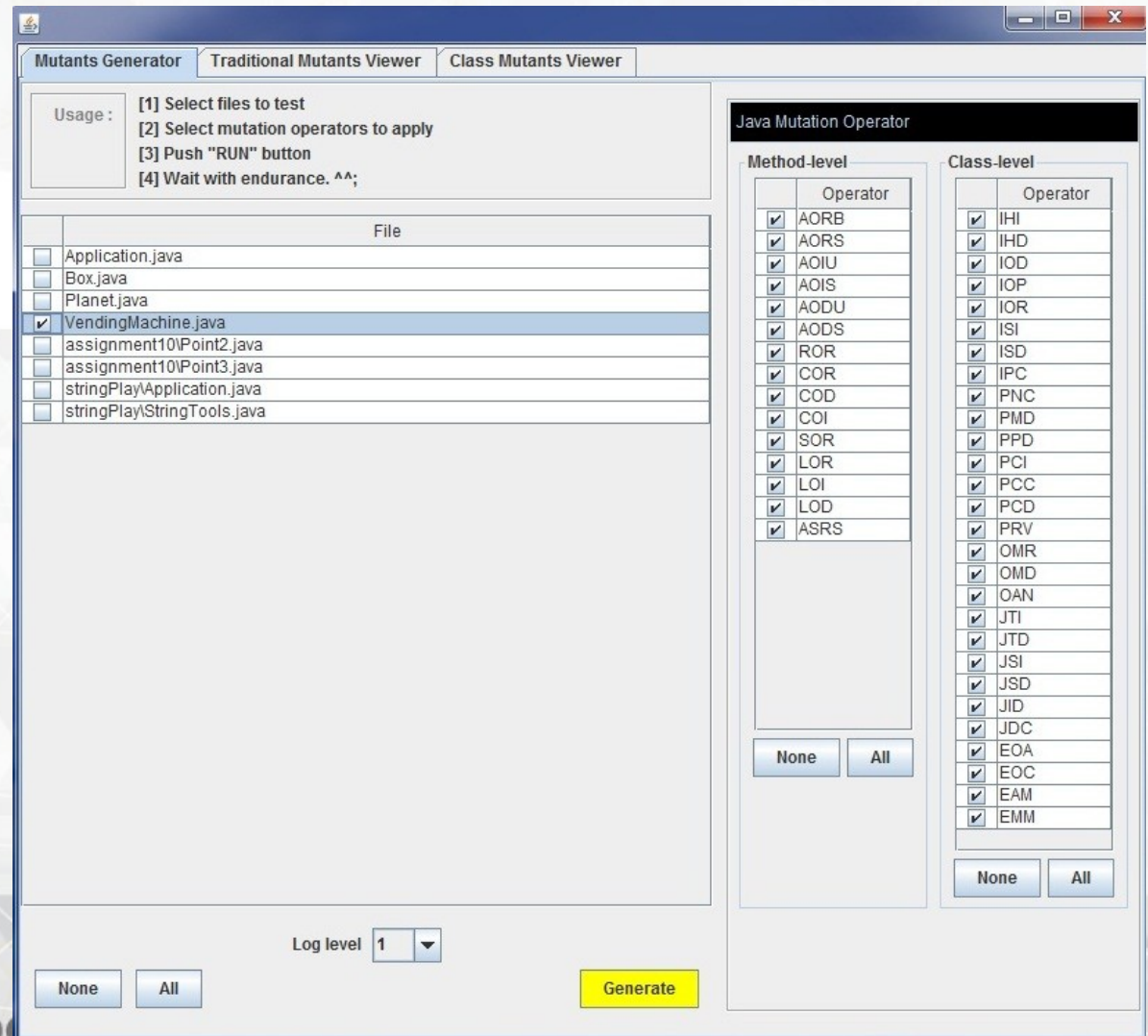
- muJava
 - No diretório
 - Salvar o(s) arquivo(s) *.java* no subdiretório *src*
 - Salvar o(s) arquivo(s) *.class* no subdiretório *classes*

Teste baseado em defeitos

- muJava
 - Uso
 - Comando no Prompt de Comando
 - `java mujava.gui.GenMutantsMain`

Teste baseado em defeitos

- muJava



Teste baseado em defeitos

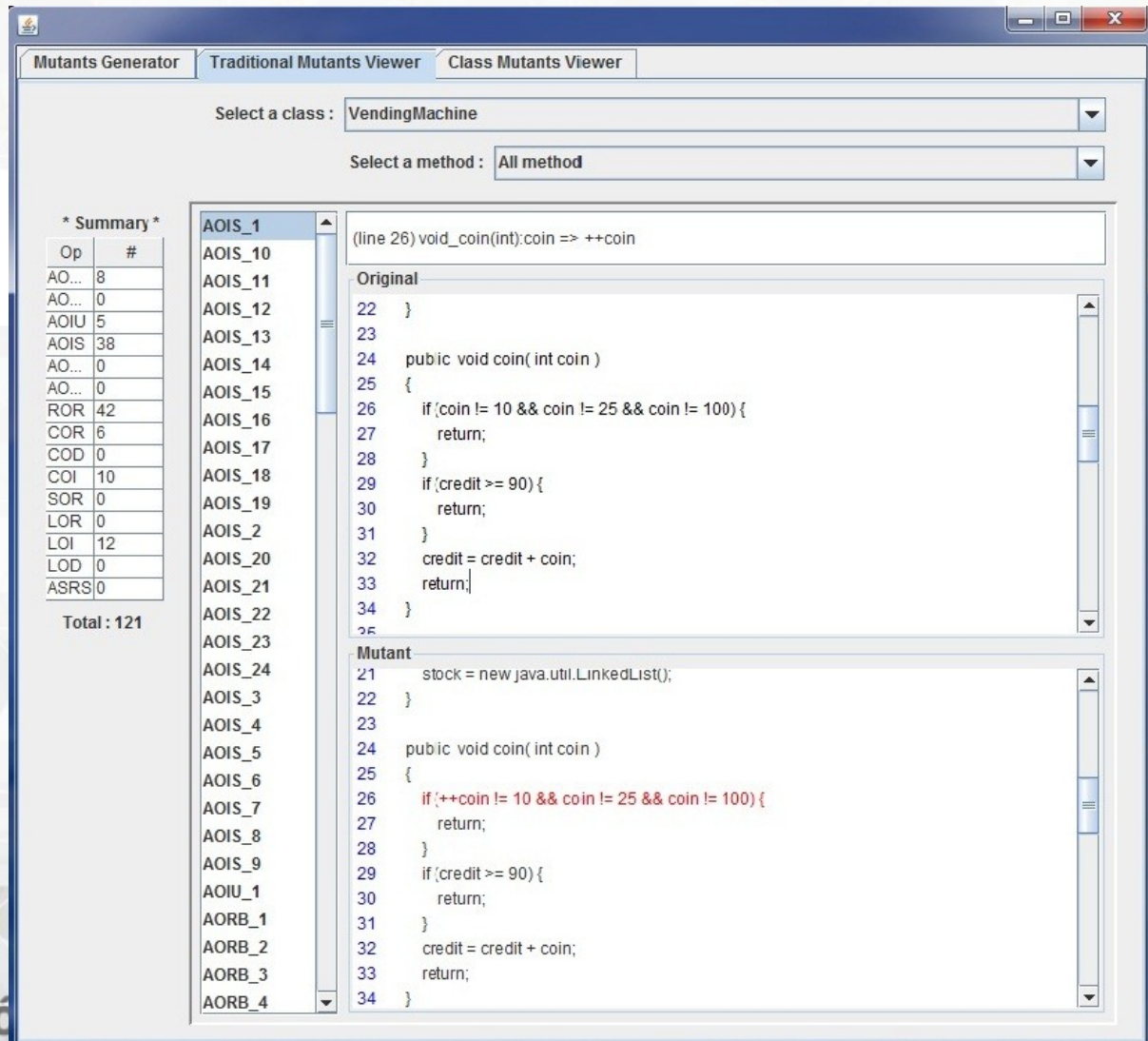
- Operadores de Mutação
 - Os operadores de mutação determinam o tipo de alteração sintática que deve ser feita para a criação dos mutantes

Teste baseado em defeitos

- Operadores de Mutação
 - AOR: *Arithmetic Operator Replacement*
 - VDL: *Variable DeLetion*
 - CDL: *Constant DeLetion*
 - ROR: *Relational Operator Replacement*
 - LOR: *Logical Operator Replacement*

Teste baseado em defeitos

- muJava



Teste baseado em defeitos

- *mutation score (ms)*
 - Equação
 - $ms(P,T) = DM(P,T) / (M(P) - ME(P))$
 - DM: mutantes mortos
 - M: total de mutantes gerados
 - ME: mutantes equivalentes
 - P: programa
 - T: casos de teste

Dúvidas